

sample content is shown below:

```
[program:your_project_name]
command = /usr/bin/python3 /path/
to/project/manage.py runserver
0.0.0.0:8000 ;
Command to start app
user = vm_user_name ;
User to run as
stdout_logfile = /path/log_file_name ;
Where to write log messages
redirect_stderr = true ;
Save stderr in the same log
environment=LANG=en_US.UTF-8,LC_ALL=en_
US.UTF-8 ;
Set UTF-8 as default encoding
```

The above *config* file instructs Supervisor what to run and where to write logs. You have to run the commands given below to make Supervisor understand the configuration for your application:

```
sudo supervisorctl reread
sudo supervisorctl update
```

The commands given below are for starting, stopping and restarting the application:

```
sudo supervisorctl start project_name
sudo supervisorctl stop project_name
sudo supervisorctl restart project_name
sudo supervisorctl status # to check
status and make sure app is running
```

Nginx configurations

We have our application running in Port 8000, which we triggered from Supervisor. Now, it is time to configure the HTTP server and redirect Port 80 requests to 8000 and also, configure SSL for our app. Make sure Nginx is running, by using the command given below:

```
sudo systemctl status nginx.service
```

You can configure your app for Nginx in the file given below:

```
/etc/nginx/sites-available/project_
```

name_here

Here is a sample configuration:

```
upstream app_server {
# fail_timeout=0 means we always retry
an upstream even if it failed
# to return a good HTTP response
# for UNIX domain socket setups
# server unix:/tmp/gunicorn.sock fail_
timeout=0;
# for a TCP configuration
server VM_IP:8000 fail_timeout=0;
# above statement will forward request
to django application which running on
port 8000
}

# below section is to forward default
request (to http port 80) to SSL port
443
server {
listen 80;
server_name host_name_or_IP_or_domain_
name www.host_name_or_IP_or_domain_
name;
return 301 https://$server_
name$request_uri;
}

server {
# use 'listen 80 deferred;' for Linux
# use 'listen 80 accept_
filter=httpready;' for FreeBSD
listen 443 ssl; # <-
ssl on; # <-
ssl_certificate /path/to/ssl/certificate;
# <-
ssl_certificate_key /path/to/ssl/
certificate_key; # <-
client_max_body_size 4G;
# set the correct host(s) for your site
server_name host_name_or_IP_or_domain_
name www.host_name_or_IP_or_domain_
name;
keepalive_timeout 5;
# path for static files
```

```
root /path/to/static/files;
location /static/ {
alias /path/to/static/files;
}
location /media/ {
alias /path/to/static/media/files/;
}

location / {
# checks for static file, if not found
proxy to app
try_files $uri @proxy_to_app;
}
location @proxy_to_app {

location @proxy_to_app {
proxy_set_header X-Forwarded-For
$proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto
https;
proxy_set_header Host $http_host;
# we don't want nginx trying to do
something clever with
# redirects, we set the Host: header
above already.
proxy_redirect off;
proxy_pass http://app_server;
}

error_page 500 502 503 504 /500.html;
location = /500.html {
root /path/to/app/current/public;
}
}
```

```
Link the site to site enabled list,
sudo ln -s /etc/nginx/sites-available/
project_name /etc/nginx/sites-enabled/
project_name
sudo service nginx restart
sudo systemctl status nginx.service
```

Now, you can access the application by its HTTPS. The HTTP request will be forwarded to HTTPS, as per the Nginx configuration. 

 By: Somanath Balakrishnan

The author has 16+ years of experience in the IT industry, and is currently working as a technology consultant at Digital Core Technologies, Kochi, Kerala.